ДЪРЖАВЕН ИЗПИТ
ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН НА ПРОФЕСИОНАЛНА КВАЛИФИКАЦИЯ –ЧАСТ ПО ТЕОРИЯ НА ПРОФЕСИЯТА

Д И П Л О М Е Н П Р О Е К Т

Тема: „Разработване на уеб базирано приложение за управление на сервиз“

професия: 481020 „Системен програмист“
специалност: 4810201 „Системно програмиране“

Дипломант: Александър Димитров Влахов, 12А клас
Ръководител на дипломния проект: инж. Веселина Стефанова Маринова

Подпис:
(дипломант)
Подпис:
(ръководител)

София, 2025 г

Съдържание

1. ВЪВЕДЕНИЕ..... 3
1.1. Задание за разработка..... 3
1.2. Анализ на заданието..... 3
1.3. Поставени цели..... 5
2. ОСНОВНА ЧАСТ..... 6
2.1. Избор на технологии..... 6
2.1.1 . Избор на програмен език..... 6
2.1.2 Избор на технология за сървъра..... 9
2.1.3 Избор на технология за базата данни..... 11
2.2. Софтуерен дизайн..... 14
2.2.1 Етапи на разработка..... 14
2.2.2 Софтуерна архитектура..... 18
2.2.3 Описание и взаимодействие с приложението..... 20
2.2.4 Графичен дизайн и структура на приложението..... 21
2.2.5 Дизайн на сървъра..... 24
2.2.6 Дизайн на базата данни..... 25
2.2.7 CRUD заявки..... 30
2.3 Тестване..... 33
3. ЗАКЛЮЧЕНИЕ..... 35
3.1 Постигнати цели..... 35
3.2 Потенциално развитие..... 36
4.ИЗПОЛЗВАНА ЛИТЕРАТУРА..... 38
5.ПРИЛОЖЕНИЯ..... 38

1. ВЪВЕДЕНИЕ

1.1. Задание за разработка

AutoExpert е иновативно уеб базирано приложение, разработено с цел да улесни управлението на автомобилни услуги и сервизи. Чрез използване на съвременни технологии като HTML, CSS, JavaScript и PHP за фронт-енд разработка, както и MySQL за управление на релационни бази данни, платформата предоставя на потребителите интуитивен интерфейс и богата функционалност.

Основната цел на AutoExpert е да осигури удобен начин за записване на часове за диагностика, прегледи и ремонти, както и да предостави възможност за подаване на заявки за спешна помощ. Платформата е предназначена за два основни типа потребители – крайни клиенти и администратори на сервиза. Докато потребителите имат достъп до записване на часове, управление на личния си профил и оставяне на отзиви, администраторите разполагат с панел за управление, чрез който могат да организират заявките, да следят историята на извършените услуги и да поддържат комуникация с клиентите.

1.2. Анализ на заданието

AutoExpert е създадено, за да отговори на нарастващата нужда от дигитализиране на автомобилния сервизен сектор. В традиционния модел на работа, клиентите трябва да посещават сервиза лично или да провеждат телефонни разговори, за да запазят час, което може да бъде неефективно и неудобно. Чрез автоматизираната система на AutoExpert тези процеси се дигитализират, като се осигурява лесен и удобен достъп до всички услуги на сервиза.

В AutoExpert всеки ще има възможност да създаде свой акаунт или да разглежда сайта като гост. Гостите обаче ще имат ограничен достъп до някои функции, като публикуване на ревюта, изпращане на съобщения и извикване на спешна помощ (пътна помощ). Системата с акаунти е създадена с цел да насърчи регистрацията на повече

потребители, като по този начин улеснява достъпа до важна информация – както за самите клиенти, така и за техните автомобили.

След регистрация, потребителите ще могат да добавят автомобили към своя профил, като всеки автомобил ще има отделна история на извършените ремонти и услуги. По този начин потребителите ще могат да следят статуса на всяка услуга – дали е в процес на изчакване

(Pending), в процес на изпълнение (Working on) или завършена (Completed). Освен това, приложението ще включва опция за записване на час за диагностика, ремонт или преглед, като потребителите ще могат да изберат конкретен автомобил, удобен за тях ден и час, при условие че няма друг предварително записан час.

За да се гарантира ефективност и оптимално разпределение на ресурсите, платформата ще забранява записване на часове в събота и неделя, както и в дните на официалните и националните празници в България. Тази мярка ще улесни работата на сервизите и ще подобри качеството на предлаганите услуги.

Допълнително, AutoExpert ще поддържа система за подаване на заявки за спешна помощ. Потребителите ще могат да изпратят заявка чрез формуляр, включващ дата и час, телефонен номер и адрес, както и опционално поле за допълнителна информация относно проблема. И тук ще важат ограниченията за записване през почивните и празничните дни, за да се гарантира адекватното управление на ресурсите.

От страна на администрацията, платформата ще предоставя панел за управление на заявки, чрез който администраторите ще могат да виждат всички активни заявки за услуги и спешна помощ. Те ще имат възможност да променят статуса на заявките – например, от "Pending" на "Working on", "Completed" или "Cancelled". Това ще улесни процеса на комуникация и организация на предоставяните услуги.

Освен основните функционалности, платформата ще включва и система за ревюта и комуникация. Потребителите ще могат да оставят отзиви за предоставените услуги, като оценяват тяхното качество. Освен това ще имат възможност да изпращат съобщения в рамките на платформата. За да се гарантира качеството на комуникацията, администраторите ще могат да преглеждат и, ако е необходимо, да изтриват неподходящи ревюта или съобщения.

1.3. Поставени цели

Основната цел на разработката на AutoExpert е създаването на иновативна и удобна за използване платформа, която да оптимизира процесите по управление на автомобилните услуги и да осигури по-добро потребителско изживяване. В този контекст, основните цели на приложението са създаване на ефективна система за управление на потребителските профили – платформата трябва да предоставя възможност за регистрация, влизане в профил и управление на лични данни и автомобили. Това ще позволи на потребителите да следят историята на своите ремонти и услуги, както и да запазват часове за бъдещи ремонти.

Оптимизиране на процеса на запазване на часове за услуги – потребителите ще могат лесно да записват час за диагностика, ремонт или преглед на автомобила си, като избират удобен за тях ден и час. Системата ще осигурява автоматичен контрол върху наличните свободни часове, като ще предотвратява записвания през уикендите и националните празници.

Интегриране на система за спешна помощ – AutoExpert ще предоставя възможност за подаване на заявки за пътна помощ чрез бърза и лесна форма, която ще включва най-важните детайли, необходими за своевременното реагиране.

Административен панел за управление на заявките – платформата ще предлага специализиран интерфейс за администраторите, чрез който те ще могат да следят всички активни заявки, да променят статуса им и да управляват комуникацията с потребителите. Това ще допринесе за по-добра организация и координация на предоставяните услуги.

Разработване на система за ревюта и комуникация – потребителите ще могат да споделят мнението си за услугите, което ще допринесе за повишаване на прозрачността и доверието към платформата. Администраторите ще имат контрол върху съдържанието на ревютата и съобщенията, като ще могат да премахват неподходящи коментари при необходимост.

Гарантиране на висока достъпност и удобство – AutoExpert ще бъде създадено с responsive дизайн, който ще осигури достъпност от различни устройства. Интерфейсът ще бъде интуитивен и лесен за навигация, за да улесни използването на платформата от потребителите.

Подобряване на ефективността и потребителското изживяване – платформата ще бъде оптимизирана за бърза работа, минимални времена за зареждане и интуитивна навигация. Това ще гарантира гладка и приятна интеракция с приложението.
Съчетаването на тези цели ще осигури надеждна, удобна и иновативна платформа, която ще улесни потребителите в управлението на автомобилните услуги. AutoExpert ще предложи ефективност, удобство и сигурност, като същевременно ще подобри цялостното взаимодействие между потребителите и сервизните центрове.

2. ОСНОВНА ЧАСТ

2.1. Избор на технологии

2.1.1 . Избор на програмен език

HTML (HyperText Markup Language) е основният език за структурирането на съдържание в уеб приложенията и страниците. Той служи като скелет, който определя структурата и организацията на уеб документите. Едно от основните предимства на HTML е неговата простота – той е лесен за изучаване и разбиране, което го прави подходящ както за начинаещи, така и за опитни разработчици. HTML е основен стандарт, разработван и поддържан от W3C, което гарантира неговата съвместимост и дългосрочна стабилност.

Една от най-важните характеристики на HTML е неговата универсалност – той работи на всички браузъри и устройства, осигурявайки широко приложение. HTML се интегрира лесно с други технологии като CSS за стилизиране и JavaScript за интерактивност, което го прави основополагащ компонент на модерния уеб. Освен това, HTML поддържа множество мултимедийни елементи, като изображения, видео и аудио, което го прави идеален за създаване на разнообразно съдържание.

Въпреки своите предимства, HTML има някои ограничения. Сам по себе си, той не предоставя динамична функционалност и изисква допълнителни технологии като JavaScript за добавяне на интерактивни елементи. Също така, без интеграция с CSS, дизайните, базирани само на HTML, често изглеждат базови и непривлекателни.

В заключение, HTML е фундаментална технология за уеб разработка, която осигурява структурирана и стандартизирана основа за изграждане на уеб приложения. Той е незаменим за създаване на уеб съдържание и осигурява стабилна основа за

интеграция с други технологии. Благодарение на своята простота, съвместимост и широка поддръжка, HTML остава ключов елемент в света на уеб разработката и продължава да бъде предпочитан избор за изграждане на уеб проекти.и уеб приложения и сайтове.

CSS (Cascading Style Sheets) е технология, използвана за стилизиране и оформление на уеб страници. Тя позволява на разработчиците да контролират визуалния вид на HTML документите, като определят цветове, шрифтове, разположения и други стилови елементи. CSS е основен инструмент за създаване на привлекателни и добре организирани уеб сайтове, който значително улеснява отделянето на структурата на съдържанието (HTML) от неговото визуално представяне.

Едно от най-големите предимства на CSS е неговата гъвкавост и възможността за централизирано управление на стиловете. С помощта на външни CSS файлове разработчиците могат да променят дизайна на множество страници едновременно, което значително улеснява поддръжката и актуализациите на уеб проектите. CSS също така предлага адаптивност чрез **Media Queries**, което го прави изключително полезен за създаване на респонсив дизайн, подходящ за различни устройства и размери на екрана.

CSS е съвместим с всички модерни браузъри и работи безпроблемно на различни платформи, което осигурява широкото му приложение. Интеграцията му с други технологии като HTML и JavaScript допълнително разширява възможностите за създаване на динамични и интерактивни уеб сайтове.

Въпреки предимствата си, CSS има и някои недостатъци. Например, сложните CSS файлове могат да станат трудни за управление, особено в големи проекти, ако не се използва добре организирана структура. Освен това, CSS няма логически конструкции като цикли и функции в основния си вид, което го прави по-малко гъвкав при сложни задачи, въпреки че препроцесори като Sass и Less решават този проблем.

В заключение, CSS е основополагаща технология за уеб разработката, която осигурява мощни инструменти за стилизиране и оформление на уеб страници. Благодарение на своята простота, съвместимост и възможности за адаптивен дизайн, CSS остава незаменим избор за създаване на модерни и визуално привлекателни уеб проекти.

JavaScript е основен език за програмиране, който играе ключова роля в съвременната уеб разработка, като позволява добавяне на интерактивност и динамично поведение на уеб страниците. Той е основен компонент за създаване на уеб приложения, които могат да реагират на действия на потребителите, да променят съдържание в реално време и да взаимодействат с външни ресурси, като API-та и бази данни. JavaScript е универсален език, който се използва както за фронтенд, така и за бекенд разработки, благодарение на платформи като Node.js, които му позволяват да работи на сървърната страна.

Едно от основните предимства на JavaScript е, че работи директно в браузъра, което позволява на уеб страниците да реагират незабавно на потребителски действия без нужда от презареждане на страницата. Това осигурява динамично и бързо взаимодействие с потребителите. JavaScript има голяма екосистема от библиотеки и фреймуъркове като React, Angular, Vue и jQuery, които значително ускоряват разработката и осигуряват готови решения за често срещани задачи.

JavaScript също така поддържа асинхронно програмиране чрез Promises и async/await, което го прави подходящ за работа с операции, които отнемат време, като зареждане на данни от сървър, без да блокират изпълнението на друг код. Тези характеристики го правят изключително полезен при изграждането на мащабируеми и ефективни уеб приложения.

От друга страна, JavaScript има и някои недостатъци. Поради динамичната си природа, той може да бъде подложен на грешки, свързани с типизацията, което понякога води до трудно откриваеми проблеми в кода. Въпреки че съвременните версии на JavaScript значително подобряват производителността и сигурността, все още има разработчици, които използват статични езици като TypeScript, за да избегнат тези проблеми. Освен това, JavaScript може да бъде тежък и да води до проблеми с производителността, ако не се използва правилно, особено при големи уеб приложения с много функционалности.

В заключение, JavaScript е не само основен език за динамично поведение в уеб страниците, но и изключително мощен инструмент за създаване на мащабируеми, ефективни и интерактивни уеб приложения. Той е основен избор за повечето уеб разработчици благодарение на своята гъвкавост, огромната си екосистема и възможността за работа както на клиента, така и на сървъра. Въпреки някои

предизвикателства, свързани с типизацията и производителността, JavaScript остава един от най-популярните и широко използвани езици за програмиране в света на уеб разработка.

2.1.2 Избор на технология за сървъра

PHP е един от най-популярните сървърни езици за програмиране, широко използван в уеб разработката. Основните му предимства включват лесната интеграция с HTML, богатата екосистема от библиотеки и рамки като Laravel и Symfony, както и съвместимостта с най-често използваните релационни бази данни като MySQL и PostgreSQL. PHP е разработен с фокус върху скоростта и удобството за разработчиците, което го прави подходящ избор за създаване на динамични уебсайтове и системи за управление на съдържанието като WordPress, Joomla и Drupal. Освен това, PHP е лесен за научаване и внедряване, тъй като повечето уеб хостинг услуги го поддържат по подразбиране. Една от основните критики към PHP е липсата на последователност в синтаксиса и по-ниската производителност в сравнение с по-модерни технологии като Node.js и Java. Въпреки това, с версиите PHP 7 и 8, езикът претърпя значителни подобрения в бързодействието, управлението на грешки и сигурността, което го прави конкурентоспособен и днес. В допълнение, PHP поддържа обектно-ориентирано програмиране, което позволява създаването на мащабируеми приложения с добра структура на кода. Също така, големият брой налични разширения и API интеграции правят PHP универсален избор за различни видове уеб приложения.

Node.js е сървърна технология, базирана на JavaScript, която използва асинхронен, неблокиращ модел на изпълнение. Това я прави изключително ефективна за обработка на множество заявки едновременно, особено в приложения в реално време като чатове, стрийминг услуги и онлайн игри. Node.js позволява използването на JavaScript както на клиента, така и на сървъра, което улеснява създаването на уеб приложения с единен технологичен стек. Благодарение на NPM (Node Package Manager), разработчиците имат достъп до хиляди модули и библиотеки, които улесняват процеса на разработка. Основните предизвикателства при използването на Node.js включват сложността на асинхронното програмиране, управлението на callback функции и потенциалните уязвимости в зависимостите на външни пакети. Освен това, макар да е ефективен за

I/O-интензивни приложения, Node.js не е оптимален за CPU-интензивни задачи, което може да ограничи употребата му в някои сценарии. Въпреки тези недостатъци, екосистемата на Node.js продължава да се развива, като предоставя инструменти като TypeScript и Express.js, които улесняват създаването на сигурни и добре структурирани приложения.

Java е утвърден език за програмиране, който е широко използван в корпоративни среди, финансови системи и приложения, изискващи висока надеждност и сигурност. Java е обектно-ориентиран език, който работи върху Java Virtual Machine (JVM), осигурявайки висока производителност и кросплатформена съвместимост. Благодарение на рамки като Spring Boot и Jakarta EE, Java предлага мощни инструменти за разработка на мащабируеми уеб приложения и микросървисни архитектури. Една от основните причини компаниите да избират Java е строгата типизация, която намалява вероятността от грешки и улеснява поддръжката на големи проекти. Въпреки своите предимства, Java има някои недостатъци, включително сложността на настройката, по-високото потребление на памет и по-дългото време за стартиране на приложенията в сравнение с по-леки решения като PHP и Node.js. Въпреки това, Java остава водещ избор за организации, които търсят стабилност, сигурност и дългосрочна поддръжка на своите софтуерни системи. Благодарение на своята мащабируемост и голямата общност от разработчици, Java продължава да бъде предпочитан език за много критични за бизнеса приложения.

След анализиране на PHP, Node.js и Java, изборът на PHP като сървърна технология е обоснован от няколко ключови фактора. PHP предлага бързо и лесно внедряване, което го прави идеален за уеб проекти, които изискват кратко време за разработка и интеграция с бази данни. В сравнение с Node.js, PHP не изисква сложни асинхронни операции и е по-подходящ за традиционни уеб приложения, които обработват заявки последователно. В сравнение с Java, PHP предлага по-опростена настройка и значително по-ниско потребление на ресурси, което го прави подходящ за хостинг на споделени сървъри и по-малки проекти. Освен това, богатата екосистема и активната общност на PHP гарантират, че разработчиците имат достъп до множество ресурси и поддръжка. Допълнително предимство на PHP е неговата гъвкавост – той може да се използва както за малки лични проекти, така и за големи корпоративни уебсайтове с висока посещаемост. Въпреки че Java и Node.js са отлични алтернативи за специфични

случаи, PHP остава оптималният избор за уеб разработка поради своята гъвкавост, ниски разходи за поддръжка и широко разпространение в индустрията. Освен това, със съвременните подобрения в езика и поддръжката на технологии като PHP-FPM и JIT компилация, PHP може да предложи конкурентна производителност, която го прави подходящ за дългосрочна употреба.

2.1.3 Избор на технология за базата данни

MySQL е една от най-популярните релационни бази данни с отворен код, която е създадена с акцент върху скоростта, стабилността и лесното използване. Тя е широко използвана в уеб разработката и е предпочитан избор за много популярни платформи като WordPress, Joomla и Magento. MySQL предлага добра производителност и е подходяща за приложения от малък до среден мащаб, както и за големи уебсайтове.

Едно от основните предимства на MySQL е, че е с отворен код и разполага както с безплатна, така и с платена версия (MySQL Enterprise), което предоставя гъвкавост при избор на лицензиране според нуждите на проекта. Тя е лесна за интеграция с различни програмни езици като PHP, Python, Java и други, което я прави подходяща за широк спектър от приложения. MySQL поддържа репликация на бази данни, което осигурява висока наличности и възможност за мащабиране. Освен това, базата данни разполага с отлична документация и голяма потребителска общност, която предоставя поддръжка и решения за често срещани проблеми. MySQL е също така съвместима с клауд-базирани решения като AWS RDS, което улеснява внедряването на модерни облачни инфраструктури.

Въпреки многото си предимства, MySQL има и някои недостатъци. Липсват някои напреднали функции, като пълна поддръжка на сложни транзакции и по-добри механизми за управление на сложни типове данни. Производителността на MySQL може да намалее при изключително големи бази данни, което я прави по-малко подходяща за мащабни корпоративни приложения. Освен това, някои функционалности, налични в платената версия MySQL Enterprise, не са достъпни в безплатната версия, което ограничава възможностите за потребители с ограничен бюджет.

MySQL е оптимален избор за уеб приложения и проекти, които изискват лесно внедряване, бързина и стабилност. Въпреки това, тя не винаги е подходяща за сложни корпоративни среди, които изискват напреднали функционалности и обработка на големи обеми данни. Тази база данни предлага баланс между простота и производителност, което я прави предпочитан избор за малки и средни проекти.

MongoDB е една от най-популярните NoSQL бази данни, която използва документи за съхранение на данни вместо традиционните таблици. Документите в MongoDB се съхраняват във формат JSON-подобни структури (BSON), което я прави изключително гъвкава за приложения с динамични или често променящи се данни. Тази база данни е проектирана за мащабируемост, висока производителност и лесно използване, което я прави предпочитан избор за модерни уеб и мобилни приложения.

Едно от основните предимства на MongoDB е нейната гъвкава структура на данните, която позволява съхранение на различни формати в един и същи документ. Това я прави подходяща за приложения, които обработват разнообразни и постоянно променящи се данни. В допълнение, MongoDB предлага хоризонтално мащабиране, което осигурява висока производителност и способност да обработва големи натоварвания чрез лесно разпределение на данните между множество сървъри. Данните в MongoDB се съхраняват във формат JSON/BSON, който е лесно четим и манипулируем от различни програмни езици като JavaScript и Python. Освен това, базата данни разполага с вградена репликация, която осигурява висока надеждност и наличност, както и отлична поддръжка за големи обеми данни и реално време, което я прави идеална за приложения като анализ на данни или модерни уеб услуги. MongoDB разполага и с широка екосистема от инструменти и интеграции, включително облачно базираната платформа MongoDB Atlas, която улеснява внедряването и управлението на проекти.

Въпреки многото си предимства, MongoDB има и някои недостатъци. Една от основните й слабости е липсата на пълна ACID поддръжка, което ограничава използването й за приложения, изискващи сложни транзакции и строга последователност на данните. Освен това, MongoDB използва повече ресурси като RAM и дисково пространство, което може да се окаже проблем за системи с ограничени ресурси. Базата данни не е оптимизирана за сложни релационни зависимости, което я прави по-малко подходяща за приложения със сложни връзки между данни. Поради

своя фокус върху производителността, MongoDB предпочита скоростта пред консистентността, което може да доведе до проблеми при критични приложения, където последователността на данните е от решаващо значение. Освен това настройването на големи клъстери може да бъде сложно и да изисква значителен опит.

MongoDB е мощен инструмент за съхранение и обработка на данни, който предлага гъвкавост и висока производителност за проекти с динамични изисквания, но трябва да се използва внимателно в ситуации, които изискват сложни транзакции или релационни зависимости.

PostgreSQL е мощна, обектно-релационна база данни с отворен код, която е известна със своята стабилност, съвместимост със стандарта SQL и богат набор от функционалности. Тя е особено подходяща за приложения, които изискват сложна обработка на данни, висока надеждност и ефективно управление на транзакции. PostgreSQL често се използва за корпоративни приложения, анализ на данни и проекти с голям обем информация, благодарение на своята висока производителност и разширяемост.

Едно от ключовите предимства на PostgreSQL е нейната поддръжка на сложни типове данни, като JSON и XML, което я прави подходяща както за релационни, така и за NoSQL приложения. Тази база данни предлага възможности за разширяване чрез персонализирани плъгини и разширения, което я прави изключително гъвкава за специализирани проекти. PostgreSQL е известна с високата си производителност при обработка на големи количества данни и сложни заявки, като същевременно поддържа ACID транзакции и гарантира строга последователност на данните. Освен това тя разполага с активна общност и редовни актуализации, които гарантират стабилност и иновации в дългосрочен план. Въпреки многобройните си предимства, PostgreSQL има и някои недостатъци. Настройката и управлението й може да са по-сложни в сравнение с MySQL, което може да изисква повече опит от администратора. Някои операции, като четене на данни за уеб приложения, могат да бъдат по-бавни в сравнение с други бази данни, особено при прости заявки. Освен това екосистемата от плъгини и инструменти е по-ограничена в сравнение с комерсиалните решения като Microsoft SQL Server, което може да изисква допълнителни усилия за внедряване на специфични функционалности.

PostgreSQL е идеален избор за сложни корпоративни приложения, които изискват висока надеждност, обработка на големи обеми данни и разширяемост.

Въпреки че е по- сложна за работа в сравнение с други бази данни, нейните функционалности и стабилност я правят предпочитана за проекти, които изискват дългосрочна ефективност и надеждност. Това я поставя като водещо решение за разработчици и компании, които търсят мощна и гъвкава база данни.

Заключение: Защо MySQL е най-добрата от трите?

MySQL се отличава като най-добрият избор от трите разгледани бази данни (MySQL, PostgreSQL и Microsoft SQL Server), когато се вземат предвид комбинацията от производителност, леснота на използване, цена и мащабируемост. Тя е идеална за уеб приложения, благодарение на своята простота, бързина и лесна интеграция с популярни технологии като PHP, Python и JavaScript. В допълнение, наличието на както безплатна версия, така и платена Enterprise версия я прави достъпна за широк кръг от потребители – от малки проекти до големи компании.

Докато PostgreSQL е по-подходяща за сложни приложения, изискващи обработка на големи данни и сложни транзакции, сложността на настройката и управлението й я прави по-малко удобна за бързо внедряване. От друга страна, Microsoft SQL Server предлага мощни инструменти за бизнес анализ, но високата цена на лицензите и зависимостта от Windows среда я правят по-малко привлекателна за малки и средни бизнеси.

MySQL, със своята стабилност, добра производителност и широка поддръжка от общността, предлага перфектния баланс между мощност и достъпност. Тя е подходяща за проекти от всякакъв мащаб, като същевременно остава лесна за настройка и управление. Именно този баланс прави MySQL най-добрият избор за разработчици и компании, които търсят надеждност, гъвкавост и ефективност без излишна сложност или високи разходи.

2.2. **Софтуерен дизайн**

2.2.1 Етапи на разработка

1. Събиране на информация и поставяне на основни цели

2. Планиране

3. Дизайн на уеб приложение

4. Кодиране и програмиране

5. Копирайтинг и етикетиране

6. Тестване, преглед и стартиране

7. Поддръжка и актуализации след стартиране

1. Събиране на информация и поставяне на основни цели

Първата стъпка при проектирането на успешен уебсайт е да се събере цялата необходима информация. На този етап трябва да се вземат предвид много фактори. Тази първа стъпка всъщност е най-важната, защото включва задълбочено разбиране на компанията, за която разработваме приложението. Предприемачите и фирмите обикновено започват с набор от идеи, когато планират да пуснат уеб приложение. Тези идеи постепенно прерастват в подробен документ, който очертава целите на приложението, неговите функции, технологичния стек, бюджета, визията и бъдещите планове. Като преглежда този документ, екипът по разработката придобива ясна представа за целите, ключовите задачи, целевата аудитория, целевата индустрия, етапите и други критични елементи на проекта. След като екипът по разработката има пълна яснота за всичко, свързано с проекта, се подготвя предложението, за да се документира всичко, което ще бъде доставено.

2. Планиране

Като използвате събраната информация от първия етап, е време да създадете план. Това включва разработването на карта на сайта (sitemap) – структуриран списък с всички основни тематични области и, ако е необходимо, техните подтеми.Картата на сайта служи като ръководство за организиране на съдържанието и осигурява последователна и интуитивна навигация. В процеса на планиране е важно да се вземат предвид крайните потребители на уебсайта – нашите клиенти. Те са тези, които ще се запознаят с нашата услуга или ще проявят интерес към нашия продукт, затова дизайнът трябва да бъде удобен и лесен за използване.

3. Дизайн на уеб приложение

Следващата стъпка е усъвършенстването на дизайна и интерактивните елементи на уеб приложението. Според предварително съгласувания план, дизайнерът създава различни макети, всеки с уникална цветова схема, шрифтове, графики, бутони, преходи и други визуални елементи. След финализирането на първоначалните макети те се представят на клиента за преглед и обратна връзка. Процесът на подобрение и итерации продължава, докато клиентът одобри крайния дизайн. Докато дизайнерският екип работи по визуалната част, екипът по разработка се фокусира върху програмната реализация. Това позволява на двата процеса да протичат паралелно, ускорявайки разработването на уеб приложението.

4. Кодиране и програмиране

На този етап уебсайтът започва да придобива завършен вид. Дизайнерът взема всички визуални елементи от одобрените макети и ги превръща в работещ уебсайт. Обикновено се започва с изграждането на началната страница, след което постепенно се добавят останалите секции и функционалности. Тук идва и техническата част – интегрират се важни елементи като система за управление на съдържанието (CMS), интерактивни форми за контакт или дори платформи за онлайн резервации. Front-end разработчиците използват технологии като HTML, CSS и JavaScript, за да създадат всичко, което потребителите виждат и с което взаимодействат. В същото време back-end екипът се грижи за логиката зад кулисите – бази данни, API-та и сървърни процеси. Този етап често включва и оптимизация за търсачки (SEO), за да може сайтът да се класира по-добре в резултатите на Google. С всяка добавена функционалност и тест, уебсайтът става все по-близо до завършена версия, готова за потребителите.

5. Копирайтинг и етикетиране

Копирайтингът и правилното етикетиране може да заемат малка част от процеса на разработка, но тяхната роля е ключова за яснотата и ефективността на цялото уеб приложение. Без добре подбрани думи и логична структура, дори най-добрият дизайн може да изглежда неинтуитивен и труден за разбиране. Доброто потребителско изживяване (UX) и интуитивният потребителски интерфейс (UI) зависят от прецизната

работа на копирайтърите и информационните архитекти. Те се грижат съдържанието да бъде не само стилно и професионално, но и лесно достъпно за потребителите. Всеки текст – от заглавията и подзаглавията до етикетите на бутоните и описанията на функциите – трябва да насочва потребителя и да улеснява навигацията в сайта. На този етап се финализират всички текстови елементи, за да бъдат не само ясни и разбираеми, но и да създадат приятно и интуитивно изживяване. Това включва оптимизация на съдържанието за по-добра ползваемост, ангажиране на потребителя и в някои случаи – подобряване на откриваемостта в търсачките. Този процес може да изглежда като малка част от цялостната разработка, но именно той често прави разликата между сайт, който просто изглежда добре, и такъв, който наистина работи ефективно за своите потребители.

6. Тестване, преглед и стартиране

Тестването е един от най-критичните етапи в процеса на разработка на уеб приложения, тъй като гарантира, че всичко работи според очакванията и отговаря на всички технически и индустриални стандарти. Без този процес рискуваме да пуснем приложение, което съдържа бъгове, проблеми със сигурността или неефективна функционалност, което би могло да доведе до лошо потребителско изживяване. Обикновено тестването се извършва от специализиран екип по осигуряване на качеството (QA), който комбинира ръчни и автоматизирани методи за откриване на грешки, дефекти и слабости в системата. Част от този процес включва валидиране на кода, за да се уверят, че приложението следва актуалните стандарти за уеб разработка и е съвместимо с различни браузъри и устройства. Освен проверката на кода, екипът по QA тества различни аспекти на софтуера – от потребителското изживяване и функционалността до производителността, сигурността и адаптивността на дизайна (responsive design). Това гарантира, че сайтът ще бъде стабилен, сигурен и удобен за работа на различни платформи, което е ключово за успеха на всяко уеб приложение.

7. Поддръжка и актуализации след стартиране

Един от най-ефективните начини да поддържаме интереса на потребителите и да привличаме нови посетители към нашия уебсайт е чрез редовно обновяване на съдържанието и разширяване на функционалностите. Независимо дали става дума за стандартен корпоративен сайт или за цялостно уеб приложение, всеки дигитален продукт изисква постоянна поддръжка, актуализации и усъвършенствания. С времето може да възникне необходимост от разширяване на услугите, добавяне на нови модули или пускане на усъвършенствана версия на приложението. Именно затова поддръжката след стартирането на проекта е толкова важна. В договора за разработка е добре да бъдат включени условия за техническа поддръжка, бъдещи подобрения и възможности за надграждане. По този начин гарантираме, че уеб приложението не само ще остане функционално и актуално, но и ще продължи да се развива в съответствие с нуждите на бизнеса и очакванията на потребителите.

2.2.2 Софтуерна архитектура

Model-View-Controller, или т.нар. MVC е архитектурен шаблон за проектиране на софтуер, който се използва за създаване на интерактивни и динамични потребителски интерфейси. За пръв път норвежкият компютърен учен Тригве Реенскауг популяризира концепцията за MVC

в края на 70-те години на XXв., докато работи по Smalltalk-80, (обектно-ориентиран език за програмиране, създаден в Xerox PARC). Накратко MVC моделът разделя приложението на три взаимосвързани части, всяка от които има отделна роля:

1.Модел (Model)- Моделът е ядрото на приложението, предопределено от областта, за която се разработва; обикновено това са данните от реалния свят, които се моделират и над които се работи – въвеждане, промяна, показване и т.н. Трябва да се прави разлика между реалния обкръжаващ свят и въображаемият абстрактен моделен свят, който е продукт на разума, който се възприема като твърдения, формули, математическа символика, схеми и други помощни средства. Например в банково приложение това са класовете, описващи клиентите, техните сметки, трансакциите, които са осъществили и т.н., както и класовете за извършване на операции над тези обекти (engines) – например клас Transfer с методи като createInterBankTransfer(), createInnerBankTransfer(), getCash() и т.н.

2.Изглед (View)- В MVC изгледът е отговорен за представянето на компонентите на потребителския интерфейс и показването на данните от модела на потребителя. Той получава данни от модела и ги подготвя за представяне. Всяко изходно представяне на информация, независимо дали е уеб страница/приложение, диаграма или схема, може да се счита за изглед. Изгледът не управлява бизнес логиката или обработката на входните данни на потребителя, а по-скоро само показва данните и компонентите на потребителския интерфейс.

3.Контролер (Controller)- Контролерът е тази част от сорс кода (клас или библиотека), която взима данните от модела или извиква допълнителни методи върху модела, предварително обработва данните, и чак след това ги дава на изгледа. Например може да бъде създаден един малък обект, в който да бъдат сложени данните за трансакцията – като в контролера бъдат взети данните за трансакцията от модела, бъдат преведени датите от UNIX формат в четим от потребителя формат, бъде преобразувана валутата от долари в евро например, бъде закръглено до втория знак вместо да се виждат данните както са в модел (и в базата) до 10-ия. Също така когато се прави уеб графичен интерфейс това би довело до много лесна модификация на HTML кода дори от човек, който не е програмист – той ще гледа на шаблона просто като на обикновена HTML страница. MVC архитектурата добавя структура и ясна логика към трислойния модел, като подобрява гъвкавостта и повторната употреба на кода. Например, моделът може да бъде използван с различни изгледи, като мобилно приложение и уеб интерфейс, без да се налага промяна в самия модел. Подобно, контролерът може да обработва различни типове взаимодействия, независимо дали идват от потребител или външна система.

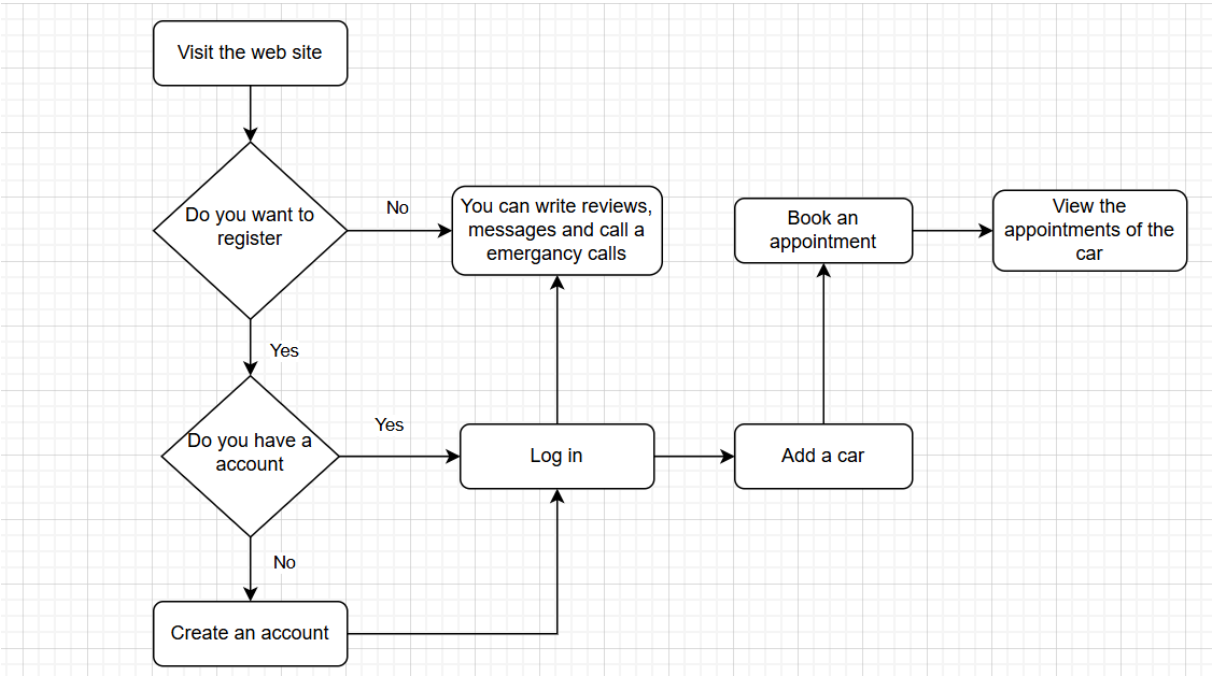
Модулността (Modularity) на архитектурата MVC е едно от основните ѝ предимства. Разделянето на отговорностите на отделни елементи (Model, View и Controller) улеснява поддръжката и разширяването. Програмата е по-управляема, когато разработчиците могат да работят поотделно върху различни нейни части, особено при големи и сложни проекти. Друго предимство е многократната употреба. Тъй като компонентите са разделени, един и същ модел може да се използва с различни изгледи или обратното, подобрявайки гъвкавостта. Тази възможност за повторна употреба насърчава ефективността на кода и спестява време за разработка в дългосрочен план. MVC моделът обаче не е лишен от своите недостатъци. Свръхсложността е една от

основните пречки. Внедряването на цялостния MVC модел в по-малки приложения може да добави ненужна сложност, което ще направи кода по-труден за четене и поддръжка.

Друг проблем възниква със синхронизацията. Тъй като компонентите са взаимозависими, осигуряването на правилна синхронизация между модела, изгледа и контролера може да бъде предизвикателство. Несъответствия в данните или състоянието могат да възникнат, ако не се управляват правилно, което води до неочаквано поведение в приложението. Освен това, кривата на обучение на MVC модела е сравнително голямо, особено за начинаещите разработчици.

2.2.3 Описание и взаимодействие с приложението

Интеракцията на всички видове потребители (регистрирани и нерегистрирани / гости) с моето уеб приложение може да се види схематично на Фиг. 2.2.3.1.



Фиг. 2.2.3.1. Data Flow Diagram на уеб приложението

Потребителите на уеб приложението могат да взаимодействат с различни негови функционалности, за да управляват своите автомобилни срещи и да използват

допълнителни опции, свързани с автомобили и услуги. За да започнат работа с уеб приложението, потребителите трябва да го посетят, като въведат неговия URL адрес или

щракнат върху линк, който води към него. След като посетят уебсайта, те могат да изберат дали искат да се регистрират.

Ако потребителят избере да не се регистрира, той ще има достъп до ограничен набор от функционалности. Те включват възможност за писане на отзиви и съобщения, както и опция за осъществяване на спешни повиквания. Това осигурява бърз и лесен достъп до основни услуги, без необходимост от създаване на профил.

Ако потребителят избере да се регистрира, но все още няма акаунт, той трябва първо да създаде такъв. Регистрацията изисква въвеждането на потребителско име, имейл адрес и сигурна парола. След успешна регистрация или ако потребителят вече има акаунт, той може да влезе в системата чрез формата за вход.

След като влезе в акаунта си, потребителят получава пълен достъп до функционалностите на уеб приложението. Той може да добави информация за своя автомобил, като въведе детайли като модел, регистрационен номер и друга важна информация.

След като автомобилът е добавен, потребителят може да резервира среща за него, като избере удобна дата и час за посещение в сервиз или друга услуга. Освен това, той може да разглежда списък с всички предстоящи и минали срещи, свързани с неговия автомобил, което улеснява управлението на ангажиментите и организирането на графика.

Уеб приложението осигурява удобен начин за управление на автомобилни срещи, като позволява на потребителите да добавят автомобили, да запазват часове и да следят своите резервации, като същевременно предоставя опции за комуникация и спешни повиквания.

2.2.4 Графичен дизайн и структура на приложението

Сега ще преминем към структурния аспект на уеб приложението AutoExpert, или с други думи казано, с помощта на какви уеб страници е изграден и структуриран той. За да може един автосервиз като този да функционира както трябва, а именно правилно,

качествено и ефективно, е необходимо той да бъде изграден от множество уеб страници. Ето от какви основни уеб страници е изграден моят уебсайт:

- Начална страница (Home / Landing Page) – Това е първата и основна страница на уебсайта, показваща се всеки път, когато потребителят влезе в уеб приложението или навигира от други страници към началната. Страницата е изградена от различни компоненти, като Него секция с приветствие и основни бутони за действие, секция за отзиви от клиенти, контактна форма и информация за местоположението и контактите на сервиза.

- Страница за регистрация (Registration Page) – Чрез тази страница потребителите могат да създадат нов профил в системата. Формата за регистрация включва задължителни полета като потребителско име, име и фамилия, имейл, телефон и парола. Системата извършва валидация на въведените данни в реално време чрез JavaScript и предотвратява дублиране на потребителски имена и имейли.

- Страница за вход (Login Page) – Позволява на регистрираните потребители да влязат в системата чрез своя имейл и парола. При успешен вход, потребителят се пренасочва към началната страница или към администраторския панел, в зависимост от неговата роля.

- Потребителски профил (Dashboard) – В тази страница потребителите могат да видят и управляват своята информация, включително:
  - Списък с техните автомобили
  - История на посещенията в сервиза
  - Възможност за запазване на час
  - Бутон за спешна помощ
  - Преглед на статуса на текущите ремонти

- Административен панел (Admin Dashboard) – Достъпен само за администратори, този панел предоставя пълен контрол над системата:
  - Управление на потребителски акаунти

- Преглед и одобрение на чакащи регистрации
- Достъп до всички системни функционалности
- Управление на часове и ремонти

- Страница за запазване на час (Book Appointment) – Позволява на потребителите да запазят час за обслужване на своя автомобил:
  - Избор на автомобил от регистрираните
  - Избор на тип услуга
  - Календар за избор на дата и час
  - Поле за описание на проблема

- Спешна помощ (Emergency Assistance) – Специална форма за спешни случаи, която включва:
  - Въвеждане на контактна информация
  - Локация за асистенция
  - Описание на проблема
  - Незабавно известяване на екипа на сервиза

-Управление на съобщения (Messages) – Централизирана система за управление на комуникацията:
  - Преглед на всички съобщения от контактната форма
  - Възможност за изтриване на съобщения
  - Сортиране по дата и важност

- История на ремонтите (Repair History) – Детайлен преглед на всички извършени ремонти:
  - Хронологичен списък на ремонтите
  - Филтриране по автомобил
  - Статус на ремонтите
  - Детайлна информация за извършените услуги

- Управление на часове (Manage Appointments) – Позволява на администраторите да:
  - Преглеждат всички запазени часове

- Променят статуса на часовете
- Филтрират по дата и статус
- Сортират по различни критерии

Всички тези страници са свързани помежду си чрез интуитивна навигация и работят заедно, за да осигурят цялостно решение за управление на автосервиз. Системата е проектирана с мисъл за лесна употреба както от клиентите, така и от администраторите, като същевременно поддържа всички необходими функционалности за ефективна работа на автосервиза.

2.2.5 Дизайн на сървъра

Сървърът е главно изграден върху PHP и MySQL, като използва PDO (PHP Data Objects) за сигурна връзка с базата данни. Но как така PHP? Той не е ли остаряла технология в сравнение с модерните JavaScript framework-ове? Да, но не съвсем. PHP продължава да бъде една от най-надеждните и широко използвани технологии за разработка на уеб приложения, особено когато става въпрос за бизнес решения като системи за управление на автосервиз.

PHP притежава т.нар. вградени сесии (Built-in Sessions) функционалност, която позволява лесно управление на потребителските сесии и автентикация. Това е особено важно за приложения като нашето, където имаме различни нива на достъп (администратори и обикновени потребители) и нужда от сигурна автентикация. Концепцията на PHP сесиите е много по-проста за имплементация в сравнение с JWT токените в Node.js например, като същевременно предоставя високо ниво на сигурност.

Освен това при PHP приложенията, всички страници се генерират динамично при заявка от потребителя (server-side rendering), а не като статични файлове. Например, когато потребител посети страницата за запазване на час, PHP първо проверява дали потребителят е автентикиран чрез сесията, след което извлича необходимите данни от MySQL базата данни (като например списък с автомобилите на потребителя) и накрая генерира HTML съдържанието, което се изпраща към браузъра.

Архитектурата на основния PHP сървър включва използването на PDO за връзка с базата данни и няколко различни PHP библиотеки. От тях най-важни са session_start()

за управление на сесиите, вградената PHP функционалност за работа с файлове, prepared statements за защита от SQL инжекции, Bootstrap за responsive дизайн и JavaScript за клиентска валидация.

За да може сървърът да работи сигурно, са имплементирани множество защити, включително хеширане на пароли с password_hash(), защита от SQL инжекции чрез prepared statements, валидация на входните данни, проверка на потребителските права и защита от Cross-Site Scripting (XSS).

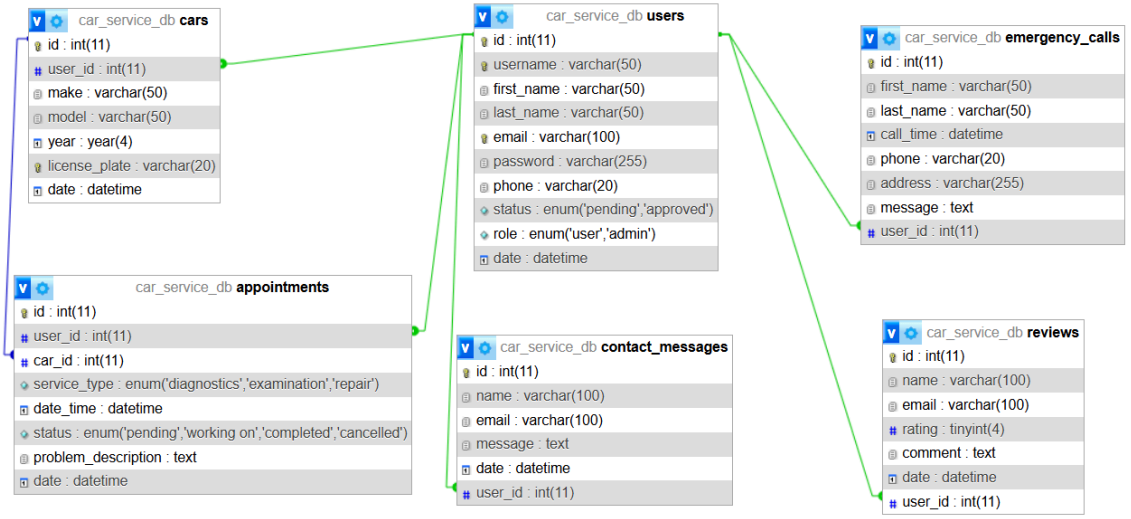
Конфигурацията на сървъра се съхранява в includes/config.php файл и включва няколко основни секции. В секцията за база данни се настройват параметрите за връзка като хост име, име на базата данни, потребителско име и парола, както и PDO опции за връзка. В секцията за сесии се конфигурират времетраенето на сесията, параметрите за сигурност и управлението на cookies. Секцията за пътища определя основната директория на проекта, директориите за качени файлове и URL адресите за пренасочване.

Системата използва MVC-подобна архитектура, където Models са PHP класове за работа с базата данни, Views са PHP файлове с HTML шаблони, а Controllers са PHP скриптове, обработващи бизнес логиката. Цялата структура е организирана в отделни директории: includes/ за конфигурационни файлове и общи функции, css/ за стилове, js/ за JavaScript файлове, images/ за изображения, както и основни PHP файлове за различните страници.

Тази архитектура позволява лесна поддръжка и разширяване на функционалността, като същевременно осигурява бързо зареждане на страниците и сигурна работа с данните на потребителите. Използването на PHP и MySQL се оказва отлично решение за този тип бизнес приложение, където надеждността, сигурността и лесната поддръжка са от първостепенно значение.

Важно е да се отбележи, че въпреки че PHP може да се счита за "по-стара" технология, тя продължава да се развива и модернизира, като предлага всички необходими инструменти за създаване на модерни и сигурни уеб приложения. В случая с AutoExpert, PHP предоставя идеалната комбинация от простота на разработка, производителност и сигурност, което го прави отличен избор за реализацията на проекта.

2.2.6 Дизайн на базата данни



Фиг. 2.2.6.1. Дизайн на базата данни

Уеб приложението за автосервиз е изградено върху релационна база данни, която съдържа шест основни таблици, всяка от които е проектирана да съхранява конкретен тип информация и да поддържа определени функционалности на системата. Базата данни е проектирана с цел да обслужва както регистрирани потребители, така и гости, осигурявайки достъп до важни функции като публикуване на отзиви, изпращане на съобщения и повикване на пътна помощ без необходимост от регистрация.

1. Таблица "users" - съхранява информация за всички регистрирани потребители на системата, като има следните колони:

- id - целочислена стойност от тип int(11), служеща като първичен ключ на таблицата
- username - съхранява потребителското име, типът на данните е varchar(50), тъй като дължината на потребителските имена може да варира
- first_name - съхранява първото име на потребителя, типът е varchar(50)
- last_name - съхранява фамилното име на потребителя, типът е varchar(50)
- email - съхранява имейл адреса на потребителя, типът е varchar(100), позволявайки съхраняването на по-дълги имейл адреси
- password - запазва хешираната парола на потребителя, типът е varchar(255), което осигурява достатъчно място за съхранение на модерни хеш алгоритми

- phone - съхранява телефонния номер на потребителя, типът е varchar(20), позволявайки включването на международни формати

- status - показва статуса на потребителя, типът е enum с възможни стойности 'pending' и 'approved', което позволява на администраторите да одобряват нови регистрации
- role - определя ролята на потребителя, типът е enum с възможни стойности 'user' и 'admin', разграничавайки обикновените потребители от администраторите
- date - съхранява датата на регистрация, типът е datetime, позволявайки точно проследяване на времевата линия

2. Таблица "cars" - съдържа информация за всички автомобили, регистрирани в системата:

- id- първичен ключ от тип int(11)
- user_id- чуждестранен ключ, свързващ автомобила с потребителя, тип int(11)
- make - марка на автомобила, тип varchar(50)
- model - модел на автомобила, тип varchar(50)
- year - година на производство, тип year(4), позволявайки съхранение на годината в стандартен формат
- license_plate - регистрационен номер, тип varchar(20), достатъчно място за съхранение на различни формати регистрационни номера
- date - дата на регистрация в системата, тип datetime

3. Таблица "appointments" - съдържа информация за записаните часове в автосервиза:

- id - първичен ключ, тип int(11)
- user_id - чуждестранен ключ към таблицата users, тип int(11)
- car_id - чуждестранен ключ към таблицата cars, тип int(11)
- service_type - тип на услугата, enum с възможни стойности 'diagnostics', 'examination', 'repair', което позволява категоризиране на различните видове обслужване
- date_time - дата и час на записания час, тип datetime
- status- статус на записания час, enum с възможни стойности 'pending', 'working on', 'completed', 'cancelled', позволявайки проследяване на хода на обслужването

- problem_description - описание на проблема, тип text, позволявайки съхранение на подробни обяснения

- date - дата на създаване на записа, тип datetime

4. Таблица "contact_messages" - съхранява съобщения от контактната форма:

- id - първичен ключ, тип int(11)
- name - име на изпращача, тип varchar(100)
- email - имейл на изпращача, тип varchar(100)
- message - съдържание на съобщението, тип text, позволявайки съхранение на дълги съобщения
- date - дата на изпращане, тип datetime
- user_id - чуждестранен ключ към таблицата users, тип int(11), който може да бъде NULL, когато съобщението е изпратено от гост

5. Таблица "emergency_calls" - съдържа информация за спешни обаждания за пътна помощ:

- id - първичен ключ, тип int(11)
- first_name - първо име на обаждащия се, тип varchar(50)
- last_name - фамилно име на обаждащия се, тип varchar(50)
- call_time - време на обаждането, тип datetime
- phone - телефонен номер, тип varchar(20)
- address - адрес за помощ, тип varchar(255), позволявайки съхранение на подробни адреси
- message - описание на спешния случай, тип text
- user_id - чуждестранен ключ към таблицата users, тип int(11), който може да бъде NULL, когато обаждането е направено от гост

6. Таблица "reviews" - съдържа отзиви от клиенти:

- id - първичен ключ, тип int(11)

- name - име на автора на отзива, тип varchar(100)
- email - имейл на автора, тип varchar(100)

- rating - оценка от 1 до 5, тип tinyint(4)
- comment - съдържание на отзива, тип text
- date - дата на публикуване, тип datetime
- user_id - чуждестранен ключ към таблицата users, тип int(11), който може да бъде NULL, когато отзивът е публикуван от гост

ВРЪЗКИ МЕЖДУ ТАБЛИЦИТЕ

Базата данни съдържа следните връзки между таблиците:

1. users → cars: Връзка от тип едно към много, тъй като един потребител може да регистрира повече от един автомобил в системата.

2. users → appointments: Връзка от тип едно към много, тъй като един потребител може да има повече от един записан час за обслужване.

3. cars → appointments: Връзка от тип едно към много, тъй като за един автомобил може да има повече от един записан час за обслужване.

4. users → contact_messages: Връзка от тип едно към много, тъй като един потребител може да изпрати повече от едно съобщение чрез контактната форма. Тази връзка е опционална, тъй като съобщения могат да бъдат изпращани и от гости.

5. users → emergency_calls: Връзка от тип едно към много, тъй като един потребител може да направи повече от едно обаждане за пътна помощ. Тази връзка е опционална, тъй като обаждания могат да бъдат направени и от гости.

6. users → reviews: Връзка от тип едно към много, тъй като един потребител може да публикува повече от един отзив. Тази връзка е опционална, тъй като отзиви могат да бъдат публикувани и от гости.

ОСОБЕНОСТИ НА ДИЗАЙНА

Уеб приложението за автосервиз е проектирано да бъде достъпно както за регистрирани потребители, така и за гости. За тази цел, таблиците contact_messages, emergency_calls и reviews са конфигурирани така, че да позволяват NULL стойности за колоната user_id, което позволява на гости да изпращат съобщения, да повикват пътна помощ и да публикуват отзиви без необходимост от регистрация.

Това проектно решение значително подобрява потребителското изживяване, тъй като позволява на клиентите да получат бърз достъп до критични услуги като пътна помощ без необходимост от предварителна регистрация. Също така, това увеличава вероятността за получаване на повече отзиви, тъй като премахва бариерата за регистрация.

Системата е проектирана с фокус върху сигурността и удобството на потребителите, като същевременно предоставя на администраторите пълен контрол върху данните и процесите в автосервиза.

2.2.7 CRUD заявки

Сега ще представя по един пример за всяка една CRUD заявка (CREATE, READ, UPDATE, DELETE), като покажа кода ми и ги обясня за какво ги използвам.

```
$sql = "INSERT INTO users (username, first_name, last_name, email, phone, password, role, status)
VALUES (:username, :first_name, :last_name, :email, :phone, :password, :role, :status)";
$stmt = $conn->prepare($sql);
$stmt->bindParam(':username', $username);
$stmt->bindParam(':first_name', $firstName);
$stmt->bindParam(':last_name', $lastName);
$stmt->bindParam(':email', $email);
$stmt->bindParam(':phone', $phone);
$stmt->bindParam(':password', $hashedPassword);
$stmt->bindParam(':role', $role);
$stmt->bindParam(':status', $status);
```

Фиг. 2.2.7.1. Създаване на запис в базата данни

Както можем да видим, Create заявката за създаване на потребителски профил е доста лесна и проста. Първо, в кода инициализираме необходимите променливи, които ще съдържат информацията за новия потребител, предоставена от формуляра. Тези полета включват потребителското име на новия потребител, първото име на

потребителя, фамилното име на потребителя, имейл адресът на потребителя, телефонният номер на потребителя, паролата на потребителя, която се хешира за

сигурност, ролята на потребителя, например "user" или "admin", и статусът на акаунта, който може да бъде "approved" или "pending". След това подготвяме SQL заявка за вмъкване на нов потребител в таблицата users, като се използват и параметри, за да предотврати SQL инжекция. Всеки параметър, например :username, :first_name и т.н., се свързва с конкретната стойност, предоставена от формуляра. След това инициализираме подготовката на SQL заявката с помощта на метода prepare() на PDO обекта, като после свързваме параметрите с реалните стойности.

```
$sql = "SELECT * FROM reviews ORDER BY date DESC";
$result = $conn->query($sql);
$reviews = $result->fetchAll(PDO::FETCH_ASSOC);

if (empty($reviews)) {
    echo "<p class='text-center'>No reviews found.</p>";
```

Фиг. 2.2.7.2. Четене на запис от базата данни

Във фиг.2.2.7.2. Read заявката за извличане на ревюта. Първо, в кода инициализираме връзката с базата данни, след което подготвяме SQL заявка, която ще извлече всички налични рецензии от таблицата reviews. Заявката избира всички колони от таблицата и ги подрежда по дата в низходящ ред, така че най-новите ревюта да се показват първи. След това подготвяме SQL заявката за изпълнение с помощта на метода query() на PDO обекта и изпълняваме заявката, за да получим резултатите. Използваме fetchAll(PDO::FETCH_ASSOC), за да извлечем всички ревюта и да ги съхраним в асоциативен масив, което улеснява достъпа до информацията.

След като сме извлекли данните, проверяваме дали има налични рецензии. Ако няма, се показва съобщение на потребителя, че няма налични ревюта. В противен случай рецензиите се визуализират в HTML формат, като за всяка рецензия се показват оценката, името на рецензента и съдържанието на рецензията. Като тази структура гарантира, че всички ревюта могат да бъдат успешно извлечени и показани на потребителите по удобен и структуриран начин.

```
$sql = "UPDATE appointments SET status = :status WHERE id = :id";
$stmt = $conn->prepare($sql);
$stmt->bindParam(':status', $newStatus);
$stmt->bindParam(':id', $appointmentId);

if ($stmt->execute()) {
    echo "<script>alert('Appointment status updated successfully!');</script>";
} else {
    echo "<script>alert('Error updating appointment status.');</script>";
}

echo "<script>window.location.href='manage_appointments.php';</script>";
```

Фиг. 2.2.7.3. Четене на запис от базата данни

Update заявката за актуализиране на статус на избраната от потребителя услуга е важна част от функционалността на нашето уеб приложение. Тази операция позволява на администраторите да променят статуса на услугата, например от "pending" на "completed".

Първо, в кода се инициализират необходимите променливи, които съдържат идентификатора на назначението и новия статус, предоставен от формуляра. След това подготвяме SQL заявка, която актуализира колоната `status` в таблицата `appointments`, като се използват параметри за предотвратяване на SQL инжекция. Всеки параметър, например `:status` и `:id`, се свързва с конкретната стойност, предоставена от формуляра. След това инициализираме подготовката на SQL заявката с помощта на метода `prepare()` на PDO обекта, като после свързваме параметрите с реалните стойности. И накрая, извикваме метода `execute()`, за да изпълним заявката и да актуализираме статуса на назначението в базата данни. Тази заявка гарантира, че статуса на назначението се актуализира само за конкретното назначение, идентифицирано по неговия уникален идентификатор. При успешна промяна на статуса, се показва съобщение за успех, а при грешка – съобщение за неуспешна актуализация. Това представлява Фиг. 2.2.7.3., описваща Update операцията, която се използва за промяна на данни в базата.

```php
$messageId = $_POST['message_id'];

$sql = "DELETE FROM contact_messages WHERE id = :message_id";
$stmt = $conn->prepare($sql);
$stmt->bindParam(':message_id', $messageId);

if ($stmt->execute()) {
    echo "<script>alert('Message deleted successfully!');</script>";
    echo "<script>window.location.href='admin_messages.php';</script>";
} else {
    echo "<script>alert('Error deleting message.');</script>";
}
```

Фиг. 2.2.7.4. Заявка за изтриване на съобщения

Във последната фигура се разглежда Delete заявката за изтриване на съобщения, която е от съществена част от функционалността на нашето уеб приложение. Тя позволява на администраторите да премахват нежелани или остарели съобщения от системата. Подобни заявки се използват и за изтриване на друга информация, която администраторът счита за ненужна или вече неактуална

Първо, в кода се инициализират необходимите променливи, които съдържат идентификатора на съобщението, предоставен от формуляра. След това подготвяме SQL заявка, която изтрива записа от таблицата contact_messages, като се използват параметри за предотвратяване на SQL инжекция. Всеки параметър, например :message_id, се свързва с конкретната стойност, предоставена от формуляра. След това инициализираме подготовката на SQL заявката с помощта на метода prepare() на PDO обекта, като после свързваме параметрите с реалните стойности.

И накрая, извикваме метода execute(), за да изпълним заявката и да изтрием съобщението от базата данни. Тази заявка гарантира, че съобщението се премахва само ако идентификаторът му съвпада с подадената стойност. При успешно изтриване на съобщението се показва съобщение за успех, а при грешка – съобщение за неуспешна операция.Това представлява четвъртата фигура, описваща Delete операцията, която се използва за премахване на данни от базата.

2.3 Тестване

Върху приложението AutoExpert е проведено обширно тестване с цел осигуряване на неговата функционалност и надеждност. За тази цел са използвани различни методи за тестване, които включват тестване на интеграцията, системата, "бялата кутия" и "черната кутия".

Тестването на интеграцията е проведено, за да се гарантира, че всички компоненти на приложението работят съгласувано. Това включва проверка на взаимодействието между различните модули като системата за запазване на часове, управлението на потребители и комуникацията с базата данни. Особено внимание е обърнато на интеграцията между формата за запазване на час и календара за наличност.

При тестването на "бялата кутия" се анализира вътрешната структура и логика на приложението. Разработчиците имат достъп до PHP кода и проучват всеки модул, за да открият потенциални грешки. Този вид тестване включва проверка на правилното хеширане на пароли, валидацията на входни данни, защитата от SQL инжекции и коректната работа на потребителските сесии.

Тестването на "черната кутия" се извършва без познание за вътрешната структура. Тестващите лица проверяват функционалността на приложението като цяло, фокусирайки се върху процеса на регистрация и вход, запазването на час за ремонт, добавянето и редактирането на автомобили, системата за спешни повиквания и прегледа на история на ремонтите.

В случаите, когато са открити проблеми, са извършени корекции като оптимизиране на SQL заявките, подобряване на валидацията на формите, добавяне на допълнителни проверки за сигурност и оптимизиране на потребителския интерфейс. След всяка корекция, тестовете са повторени, за да се гарантира, че промените не са засегнали други функционалности.

Специално внимание е обърнато на тестването на системата за спешни повиквания, тъй като тя е критична за бизнеса и трябва да работи безотказно 24/7.

Всички тези тестове помагат да се осигури високо качество на услугата и добро потребителско изживяване, което е от ключово значение за успеха на система за управление на автосервиз.

3. ЗАКЛЮЧЕНИЕ

3.1 Постигнати цели

Разработването на уеб базирано приложение за управление на автосервиз, на име AutoExpert, бе успешно осъществено, като бяха постигнати абсолютно всички предварително зададени критерии и цели.

През целия процес на разработка на активен фокус беше създаването на иновативно и адаптивно уеб приложение, което да предложи на потребителите най-доброто потребителско изживяване, независимо от техните устройства. Със специално внимание към детайлите и функционалностите, успешно се разработи уеб платформа с чист, разбираем и лесен за поддръжка PHP код, която не само оптимизира функционирането и разширяването на приложението, но и предоставя на потребителите удобен и интуитивен интерфейс за лесна навигация и приятно изживяване.

Успешно бяха създадени и имплементирани всички необходими и зададени функционалности. Системата за управление на потребителски профили позволява лесно създаване и управление както на клиентски, така и на администраторски акаунти, като всеки тип потребител има достъп до специфични за ролята си функции. Функционалността за запазване на час е интуитивна и позволява на клиентите бързо и лесно да планират посещенията си в сервиза, независимо дали става въпрос за рутинна диагностика или спешен ремонт.

Особено внимание бе отделено на системата за проследяване историята на ремонтите, която дава възможност на клиентите да имат пълен поглед върху извършените дейности по техните автомобили. Тази функционалност не само улеснява клиентите, но и помага на сервиза да поддържа детайлна документация за всеки автомобил.

Една от най-важните функционалности, която беше успешно имплементирана, е системата за спешна помощ. Тя позволява на клиентите бързо и лесно да извикат пътна помощ при нужда, като системата автоматично изпраща локацията и детайлите за

проблема към екипа на сервиза. Тази функция е особено ценна в критични ситуации и значително подобрява качеството на обслужване.

Накрая, но не на последно място, беше разработена система за отзиви и оценки, която дава възможност на клиентите да споделят своето мнение за получените услуги. Тази функционалност не само помага на бъдещите клиенти да направят информиран избор, но и предоставя ценна обратна връзка на сервиза за качеството на предлаганите услуги.

Всички тези функционалности работят заедно, за да създадат цялостно решение, което отговаря на нуждите както на клиентите, така и на персонала на автосервиза. Платформата успешно съчетава удобство на използване с надеждност и ефективност, като същевременно предоставя всички необходими инструменти за успешното управление на един модерен автосервиз.

3.2 Потенциално развитие

Въпреки че приложението е напълно функционално и работи ефективно като система за управление на автосервиз, съществуват възможности за бъдещо разширение и подобрение на функционалностите. Тези подобрения биха направили системата още по-ефективна и удобна за използване както от клиентите, така и от персонала на автосервиза.

В областта на разширяването на услугите, системата може значително да се надгради. В момента тя предлага основни автосервизни услуги, но има потенциал за интеграция със система за поръчка на авточасти, което би улеснило както сервиза, така и клиентите при необходимост от подмяна на части. Може да се добави модул за проследяване на доставки на части, който да информира както сервиза, така и клиентите за статуса на поръчаните компоненти. Автоматичното известяване за

предстоящи технически прегледи би било полезно допълнение, помагащо на клиентите да поддържат автомобилите си в изправност. Система за лоялни клиенти с точки и отстъпки би насърчила редовното използване на услугите на сервиза, а интеграцията с GPS системи би подобрила значително услугата за спешна помощ на пътя.

По отношение на комуникацията, системата може да се надгради с няколко ключови функционалности. Внедряването на чат система в реално време между клиенти и сервиз би улеснило комуникацията и би позволило бързи консултации. Автоматичните SMS известия за статуса на ремонта биха държали клиентите информирани без нужда от постоянно проверяване на системата. Интеграцията с популярни приложения за съобщения като WhatsApp и Viber би направила комуникацията още по-удобна, а възможността за видео консултации би позволила на механиците да дават първоначални насоки дори преди автомобилът да пристигне в сервиза. Системата за автоматични напомняния за профилактика би помогнала за превантивната поддръжка на автомобилите.

Функционалността за плащане също може да се разшири значително. Добавянето на повече опции за онлайн плащане, включително ePay, PayPal и дори криптовалути, би направило разплащанията по-удобни за различните типове клиенти. Система за разсрочено плащане би направила по-скъпите ремонти по-достъпни за клиентите. Интеграцията с фактуриращ софтуер и автоматичното генериране на фактури би оптимизирало административната работа, а програма за лоялност с бонус точки би насърчила редовното използване на услугите на сервиза.

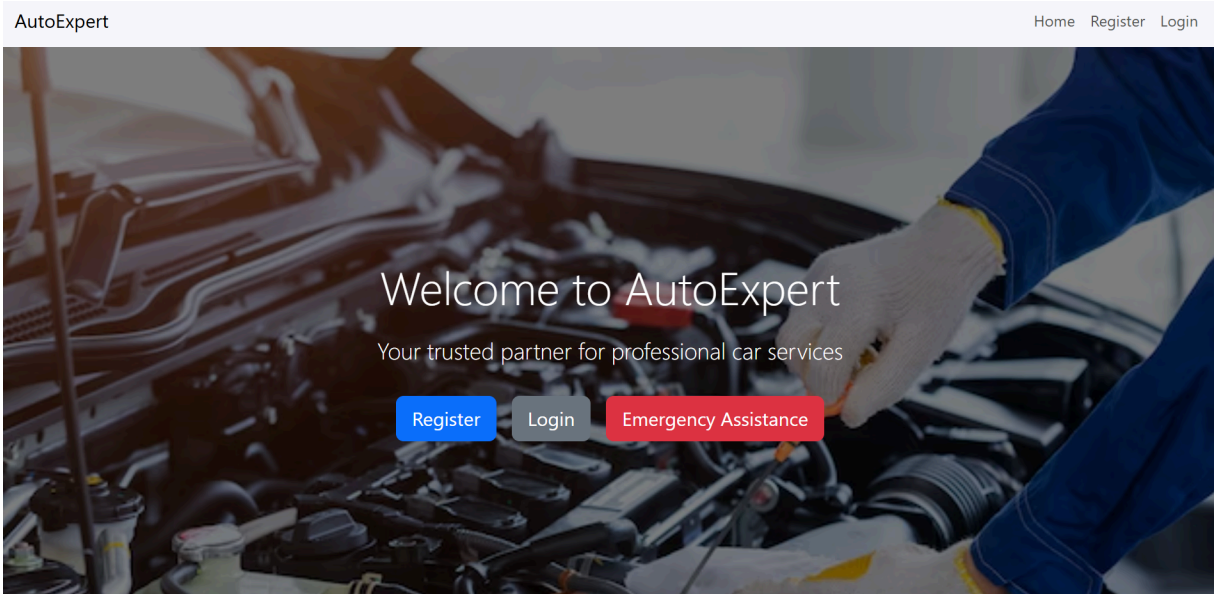
Потребителският интерфейс също има потенциал за подобрение. Разработването на мобилно приложение за iOS и Android би направило системата още по-достъпна за клиентите. По-детайлната визуализация на ремонтния процес би дала на клиентите по-добра представа за извършваната работа. Интерактивна диагностична система би помогнала за по-бързото идентифициране на проблеми, а подобреният календар за запазване на часове би направил планирането на посещения в сервиза още по-лесно. Персонализираните известия според предпочитанията на потребителя биха осигурили по-добро потребителско изживяване.

Всички тези подобрения са планирани с мисъл за подобряване на цялостното изживяване на клиентите и оптимизиране на работата на сервиза. Приоритет ще бъде даден на функционалностите, които директно подобряват потребителското изживяване и улесняват работата на сервиза, като постепенно се внедряват и останалите планирани подобрения.

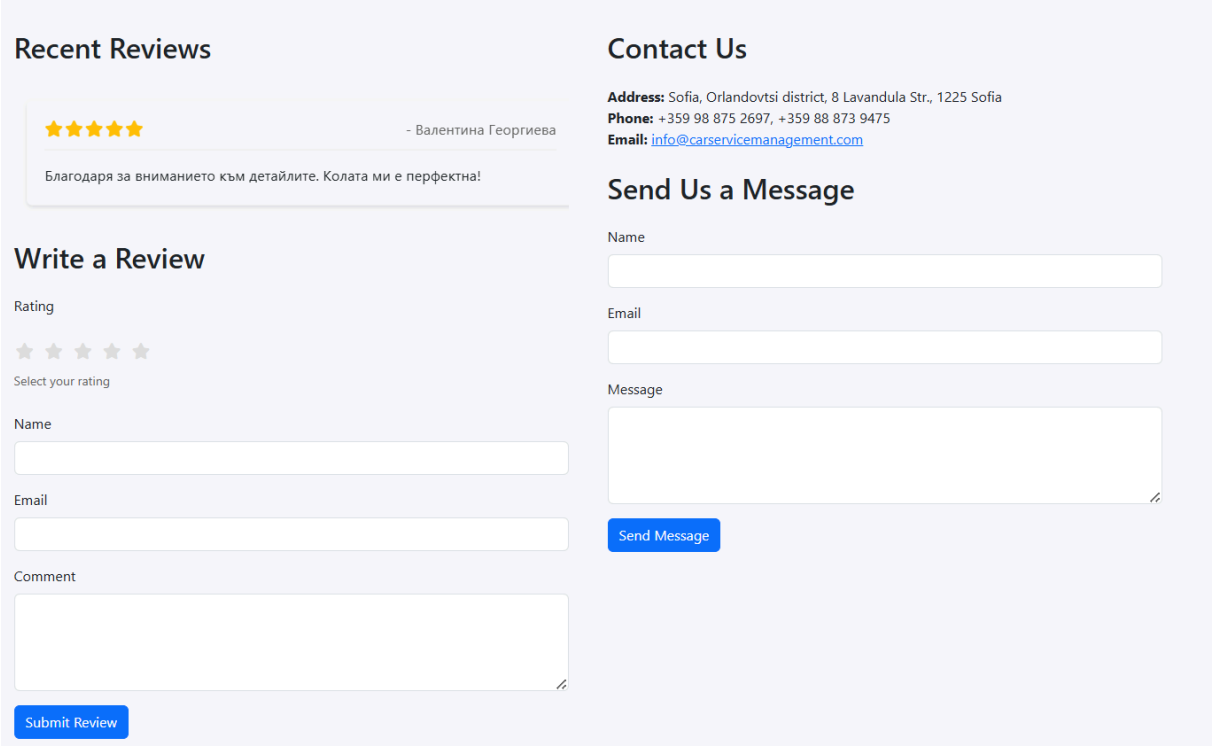
4.ИЗТОЧНИЦИ

1. Основни характеристики и описание на MySQL: https://en.wikipedia.org/wiki/MySQL?spm=a2ty_o01.29997173.0.0.44f44997ZVW2Zt
2. HTML Tutorial: https://www.w3schools.com/html/?spm=a2ty_o01.29997173.0.0.44f44997bCcGHJ
3. JavaScript Tutorial: https://javascript.info/?spm=a2ty_o01.29997173.0.0.44f44997bCcGHJ
4. Описание и анализ на сървърните технологии като цяло: https://www.studysmarter.co.uk/explanations/computer-science/computer-network/server-side-technologies/?spm=a2ty_o01.29997173.0.0.44f44997bCcGHJ
5. Описание на софтуерната архитектура MVC: https://www.geeksforgeeks.org/mvc-framework-introduction/?spm=a2ty_o01.29997173.0.0.44f44997ZVW2Zt
6. Видео за Responsive Design с CSS: https://www.youtube.com/watch?spm=a2ty_o01.29997173.0.0.44f44997ZVW2Zt&v=yfoY53QXEnI
7. Видео за MVC Архитектура с PHP: https://www.youtube.com/watch?spm=a2ty_o01.29997173.0.0.44f44997ZVW2Zt&v=4ZmYgoUdoWQ
8. Web App Development Process Model: https://www.techosquare.com/blog/web-app-development-process-model?spm=a2ty_o01.29997173.0.0.44f44997ZVW2Zt
9. ChatGPT: https://chat.openai.com/?spm=a2ty_o01.29997173.0.0.44f44997ZVW2Zt

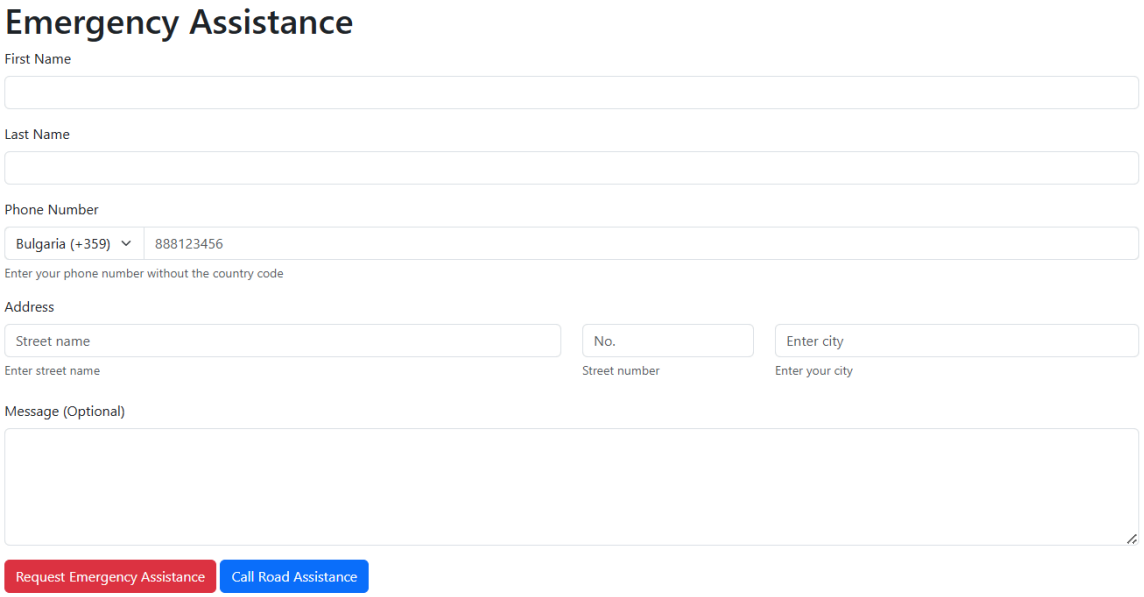
5.ПРИЛОЖЕНИЯ



Фиг. 5.1. Част от началната страница на приложението



Фиг. 5.2. Част от началната страница, в която се публикуват ревюта и съобщения



Фиг. 5.3. Страница за извикване на спешна помощ (пътна помощ) при нужда

Фиг. 5.4. Страница за запазване на час за диагностика, преглед и ремонт

Repair History

Selected Car Details

Make: Mercedes-Benz

Model: C-Class

Year: 2019

License Plate: P9823BA

Service History for Selected Car

Service Type	Date and Time	Status	Description
Examination	2025-06-19 12:20:00	pending	Рутинен преглед-хидравлична система
Repair	2025-05-05 11:50:00	pending	Смяна на свещи и филтри на двигателя
Repair	2025-04-18 14:30:00	pending	Ремонт на вратите-замяна на електрическия механизъм

Фиг. 5.5. Страница за преглед на историята на автомобил

Admin Dashboard

Welcome, Super Admin!

No pending user accounts.

Manage Users | Messages | All Reviews | Emergency Calls | Manage Appointments | Logout

Фиг. 5.6. Admin dashboard